

Имитационное моделирование

Лабораторная работа № 7. Основные дискретно-событийные модели

Исаева Зарина Исмайлбековна

2026-09-07

Содержание

- 1 1 Информация
- 2 2 Дискретные события
- 3 3 Очередь М/М/с
- 4 4 Система Росса
- 5 5 Проверка и результат

Раздел 1

1 Информация

1.1 Докладчик

- Исаева Зарина Исмайлбековна
- студенческий билет: **1132232882**
- дисциплина: «Имитационное моделирование»
- лабораторная работа № 7

1.2 Тема

Реализация основных моделей в дискретно-событийном подходе

Две системы:

- очередь М/М/с;
- резервируемая система Росса.

1.3 Цель

Реализовать процессные дискретно-событийные модели в Julia и проверить имитационные оценки аналитическими характеристиками.

1.4 Задачи

1. Воспроизвести базовую M/M/2.
2. Исследовать влияние λ и c .
3. Воспроизвести систему Росса.
4. Добавить несколько ремонтников и разные N .
5. Сопоставить имитацию и теорию.

Раздел 2

2 Дискретные события

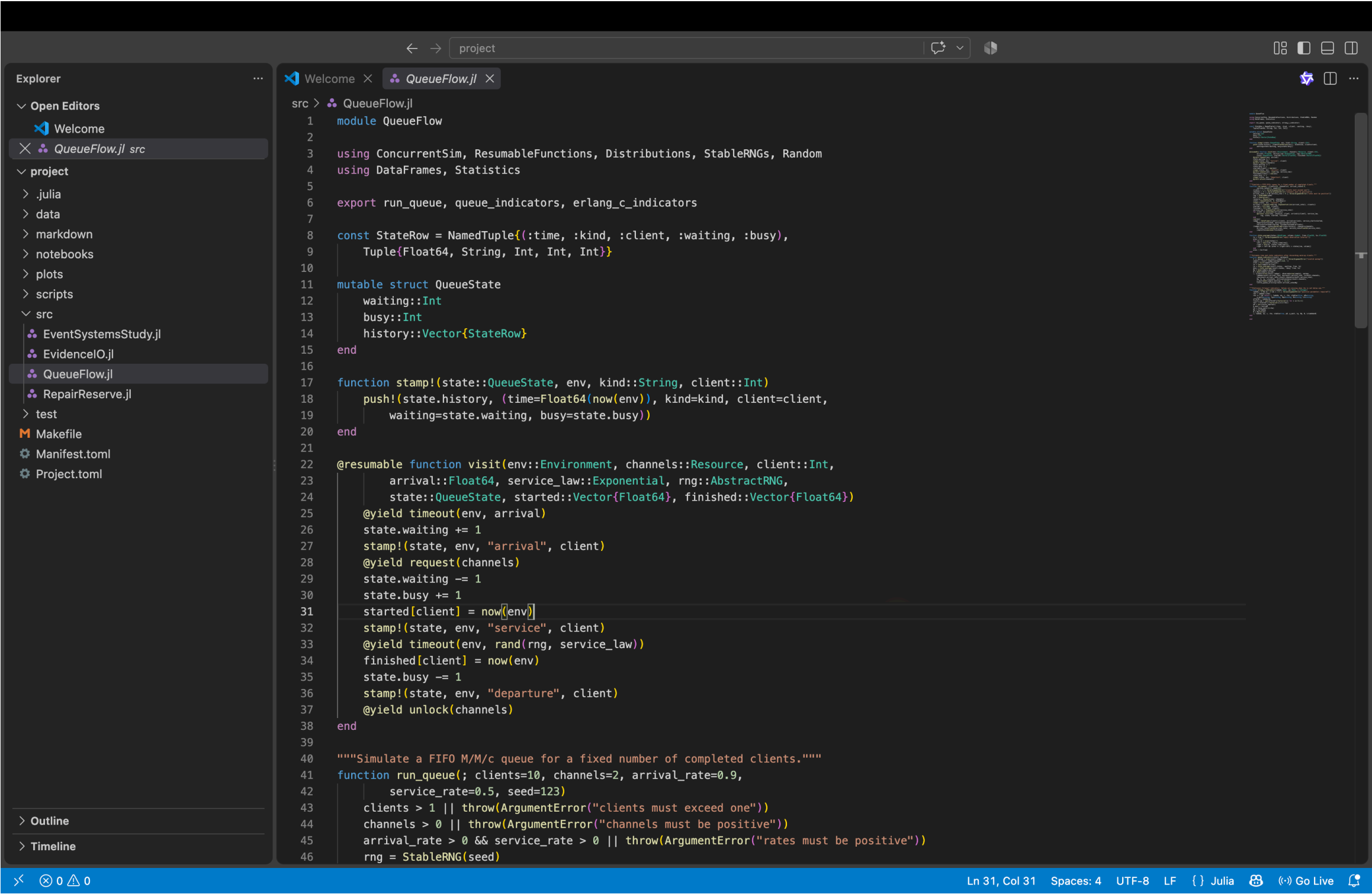
2.1 Календарь событий

- состояние меняется только в моменты событий;
- `Simulation` хранит модельное время;
- `timeout` планирует следующее событие;
- `Resource` ограничивает число каналов;
- `@resumable` приостанавливает и продолжает процесс.

2.2 Структура проекта

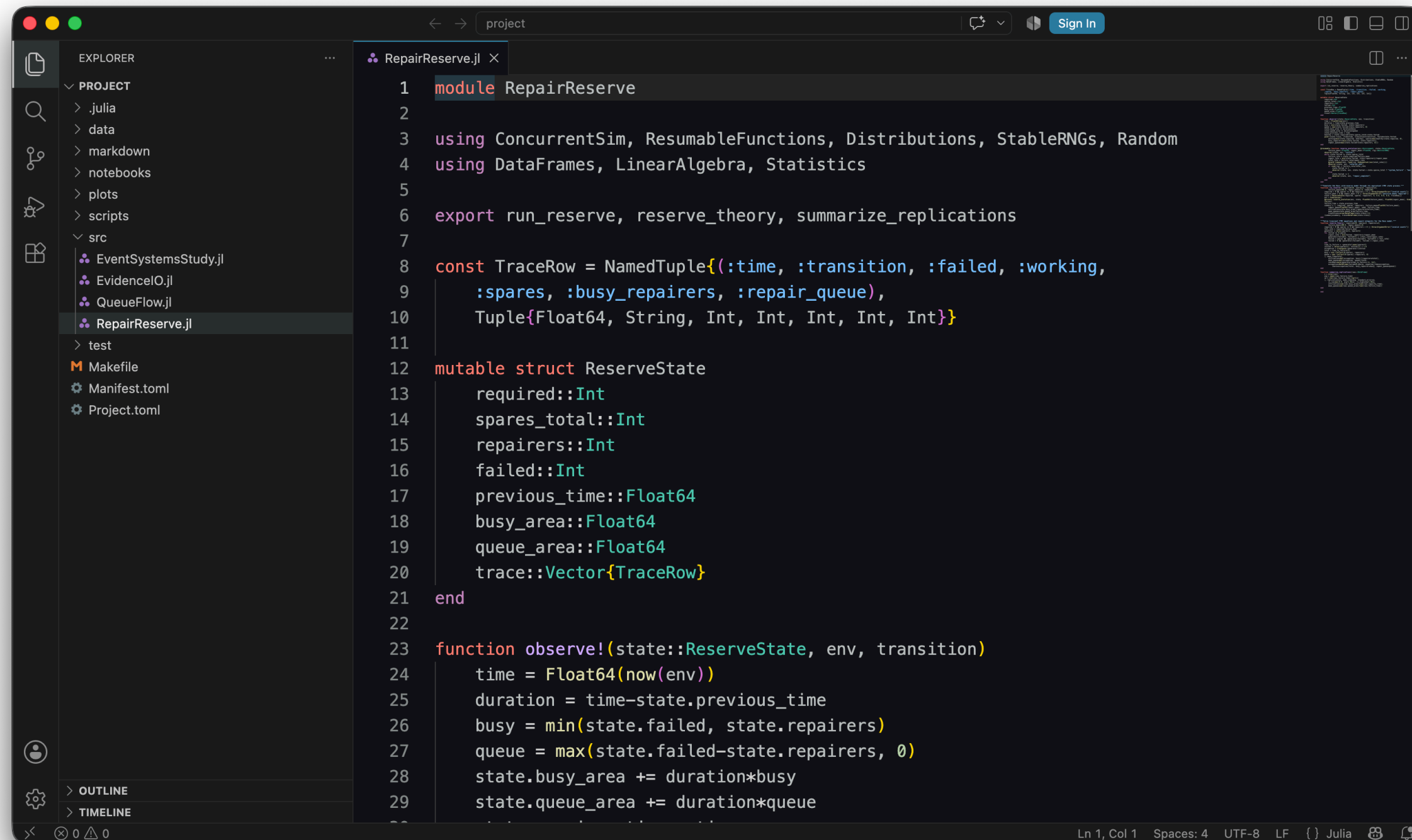
- `src` — модели и сохранение результатов;
- `scripts` — четыре литературных сценария;
- `data, plots` — наблюдения;
- `notebooks` — выполненные IPYNB;
- `markdown` — QMD и автономные HTML.

2.3 Настоящий VS Code



Модуль очереди и структура проекта

2.4 Второй модуль в VS Code



Раздел 3

3 Очередь М/М/с

3.1 Предпосылки

- пуассоновский поток с интенсивностью λ ;
- экспоненциальное обслуживание с интенсивностью μ ;
- с параллельных каналов;
- FIFO;
- стационарность при $\rho < 1$.

3.2 Коэффициент загрузки

$$\rho = \frac{\lambda}{c\mu}$$

Для базовой М/М/2:

$$\rho = \frac{0,9}{2 \cdot 0,5} = 0,9.$$

Система устойчива, но работает близко к пределу.

3.3 Формула Эрланга С

$$P_{wait} = \frac{(c\rho)^c}{c!(1-\rho)} P_0$$

$$L_q = \frac{\rho}{1-\rho} P_{wait}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}.$$

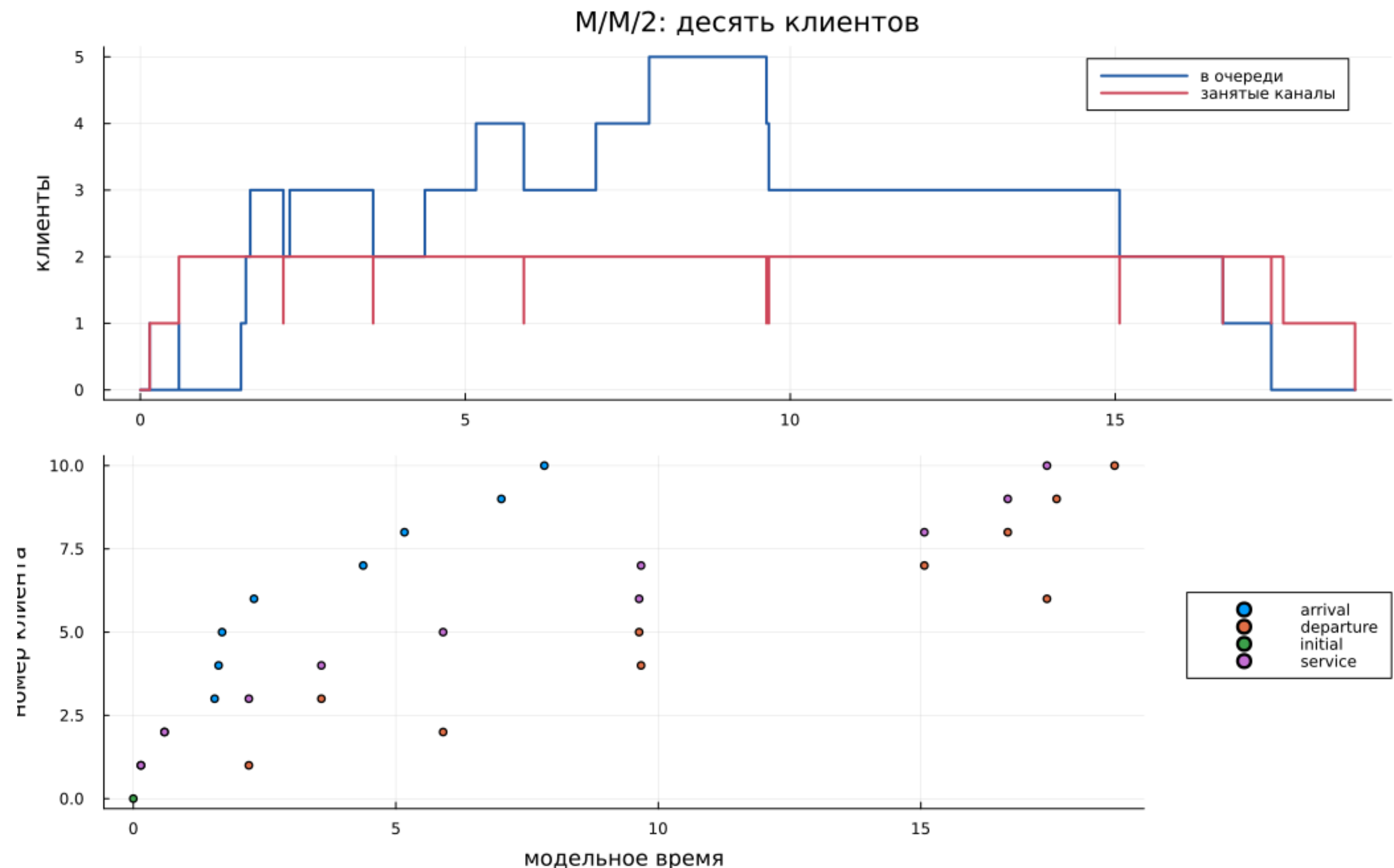
3.4 Базовые параметры

Параметр	Значение
Клиенты	10
Каналы <i>c</i>	2
λ	0,9
μ	0,5
Seed	123

3.5 Процесс клиента

```
1 @yield timeout(env, arrival)
2 state.waiting += 1
3 @yield request(channels)
4 state.waiting -= 1
5 state.busy += 1
6 @yield timeout(env, rand(rng, serviceLaw))
7 state.busy -= 1
8 @yield unlock(channels)
```

3.6 Базовая траектория



Очередь и занятые каналы

3.7 Короткий и стационарный опыт

Источник	W_q	W	P_{wait}
10 клиентов	4,8568	8,4105	0,8000
Эрланг С	8,5263	10,5263	0,8526

Десять клиентов показывают механизм, но не стационарное среднее.

3.8 Выполненный notebook M/M/2

Базовая очередь M/M/2

Воспроизводится пример методических указаний: десять клиентов, два канала, интенсивности поступления и обслуживания 0,9 и 0,5, генератор StableRNG(123).

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(srcdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.QueueFlow, .EventSystemsStudy.EvidenceIO
using DataFrames, Plots
```

Имитационный прогон

```
result = run_queue(clients=10, channels=2, arrival_rate=0.9,
                  service_rate=0.5, seed=123)
observed = queue_indicators(result)
theory = erlang_c_indicators(0.9, 0.5, 2)
store_table("queue_baseline", "clients.csv", result.ledger)
store_table("queue_baseline", "states.csv", result.states)
store_table("queue_baseline", "comparison.csv", DataFrame([
    (source="simulation", Wq=observed.Wq, W=observed.W, Lq=observed.Lq,
     L=observed.L, p_wait=observed.p_wait),
    (source="Erlang C", Wq=theory.Wq, W=theory.W, Lq=theory.Lq,
     L=theory.L, p_wait=theory.p_wait)]))
```

"/workspace/labs/lab87/project/data/queue_baseline/comparison.csv"

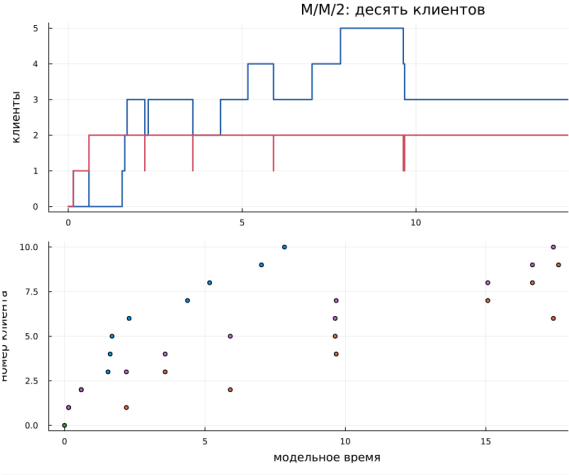
События и результат

```
println(first(result.ledger, 10))
println("completion = ", round(result.completion, digits=4))
println("rho = ", theory.rho, ", theoretical Wq = ", round(theory.Wq, digits=4))
figure = queue_state_plot(result.states; title="M/M/2: десять клиентов")
store_figure("queue_baseline", "queue_timeline.png", figure)
figure
```

10×7 DataFrame

Row	client	arrival	service_start	departure	waiting	service	system
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	1	0.145185	0.145185	2.20099	0.0	2.0558	2.0558
2	2	0.594183	0.594183	5.90072	0.0	5.30654	5.30654
3	3	1.54906	2.20099	3.58221	0.051921	1.38123	2.03315
4	4	1.62428	3.58221	9.67069	1.95793	6.08848	8.04641
5	5	1.6911	5.90072	9.6342	4.20962	3.73347	7.9431
6	2	2.2989	9.6342	17.4018	7.33529	7.76764	15.1029
7	4	3.7793	9.67069	15.067	5.29276	5.39629	10.609
8	8	5.16404	15.067	16.6555	9.90204	1.58857	11.4906
9	9	7.00999	16.6555	17.5861	9.64555	0.930517	10.5761
10	10	7.82899	17.4018	18.6903	9.57284	1.28843	10.8613

completion = 18.6903
rho = 0.9, theoretical Wq = 8.5263

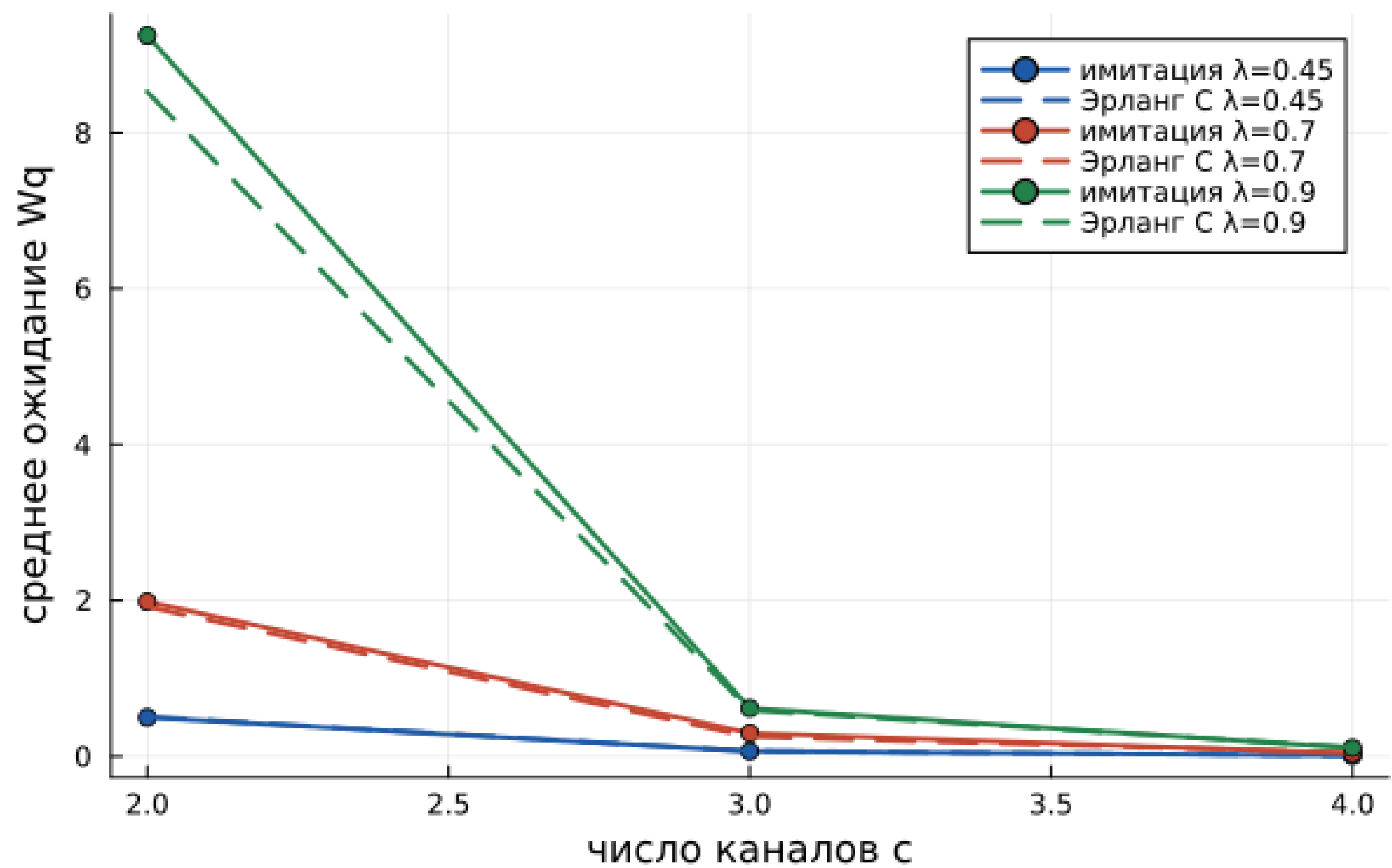


This notebook was generated using [Literate.jl](#).

3.9 План параметрического опыта

- $\lambda \in \{0,45; 0,70; 0,90\}$;
- $c \in \{2; 3; 4\}$;
- $\mu = 0,5$;
- 15 000 клиентов;
- разогрев 3 000 клиентов.

Имитация и формула Эрланга С



Девять устойчивых режимов

3.11 Вывод по очереди

- увеличение c резко сокращает W_q ;
- при $c = 2$ рост λ до 0,9 создаёт большую очередь;
- максимальная относительная ошибка W_q — 19,62%;
- остаток закона Литтла мал по сравнению с масштабом показателей.

Раздел 4

4 Система Росса

4.1 Схема

- N машин постоянно должны работать;
- S машин находятся в холодном резерве;
- отказавшие машины ремонтируются;
- отказ системы: число неисправных стало $S + 1$.

4.2 Интенсивности СТМС

В состоянии с k неисправными машинами:

$$q_{k,k+1} = \frac{N}{F}, \quad q_{k,k-1} = \frac{\min(k, r)}{G}.$$

Состояние $S + 1$ — поглощающее.

4.3 Базовые параметры

Параметр	Значение
Рабочие машины N	10
Резерв S	3
Ремонтники r	1
Среднее до отказа F	100 ч
Среднее ремонта G	1 ч
Прогоны	5

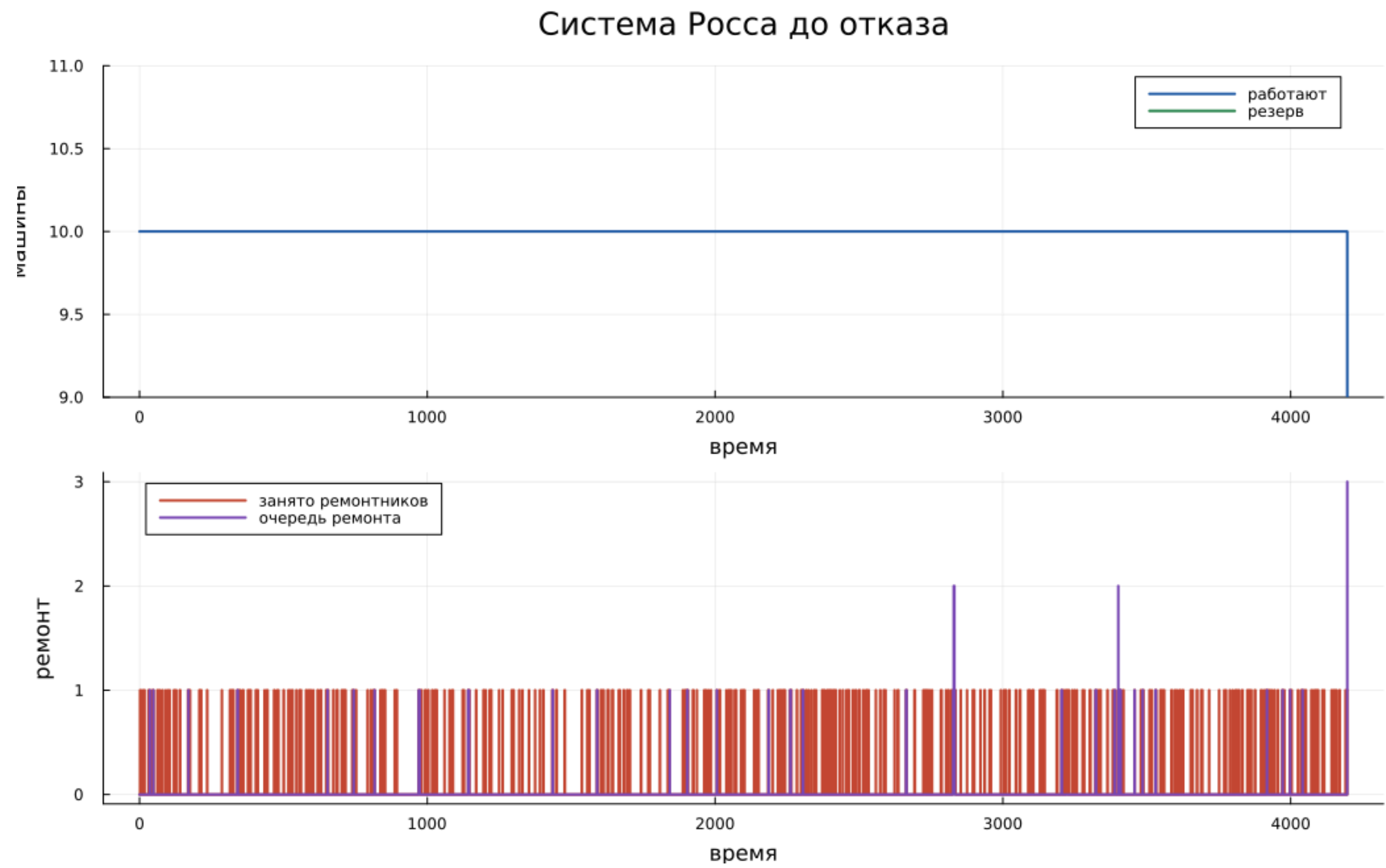
4.4 Эквивалентная событийная реализация

```
1 failure_rate = required/failure_mean
2 repair_rate = min(failed, repairers)/repair_mean
3 total_rate = failure_rate + repair_rate
4 @yield timeout(env, rand(rng, Exponential(inv(total_rate))))
5 failed += rand(rng) < failure_rate/total_rate ? 1 : -1
```

4.5 Почему агрегация точна

- времена экспоненциальны;
- будущая интенсивность зависит только от k ;
- личности машин не входят в состояние;
- календарь ConcurrentSim сохраняется;
- число событий уменьшается без изменения распределения отказа.

4.6 Траектория до отказа



4.7 Пять прогонов

№	Время отказа, ч
1	4 196,93
2	22 311,63
3	66 946,37
4	12 198,35
5	31 867,14

4.8 Теоретическое решение

$$Q_t = -1$$

- аналитическое среднее: **12 340 ч**;
- невязка: меньше 10^{-9} ;
- загрузка одного ремонтника: **0,09968**;
- средняя очередь ремонта: **0,01053**.

4.9 Почему среднее пяти прогонов отличается

- отказ с тремя резервами — редкое событие;
- распределение времени имеет большой разброс;
- один длинный прогон сильно меняет среднее;
- аналитическое значение остаётся контрольным;
- параметрический опыт использует больше повторов.

4.10 Выполненный notebook Росса

Базовая резервируемая система Росса

Сохранены параметры примера: десять рабочих машин, три холодных резервных, один ремонтник, средние времена отказа и ремонта 100 и 1 час, пять прогонов.

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwttype"] = "100"
include(scrdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.RepairReserve, .EventSystemsStudy.EvidenceIO
using DataFrames, Statistics, Plots
```

Пять воспроизводимых прогонов

```
runs = NamedTuple[]
traces = DataFrame[]
for run in 1:5
    result = run_reserve(required=10, spares=3, repairers=1,
        failure_mean=100.0, repair_mean=1.0, seed=149+run)
    push!(runs, merge((runrun,), result.summary))
    push!(traces, result.trace)
end
run_table = DataFrame(runs)
first_trace = first(traces)
theory = reserve_theory(required=10, spares=3, repairers=1,
    failure_mean=100.0, repair_mean=1.0)
comparison = DataFrame(observed_mean=[mean(run_table.failure_time)],
    analytical_mean=[theory.mean_time], observed_utilization=[mean(run_table.utilization)
    analytical_utilization=[theory.utilization], observed_queue=[mean(run_table.mean_queue)
    analytical_queue=[theory.mean_queue])
store_table("reserve_baseline", "five_runs.csv", run_table)
store_table("reserve_baseline", "first_trace.csv", first_trace)
store_table("reserve_baseline", "analytical_comparison.csv", comparison)
store_table("reserve_baseline", "state_occupation.csv", theory.occupation)
```

"/workspace/labs/lab07/project/data/reserve_baseline/state_occupation.csv"

Итог

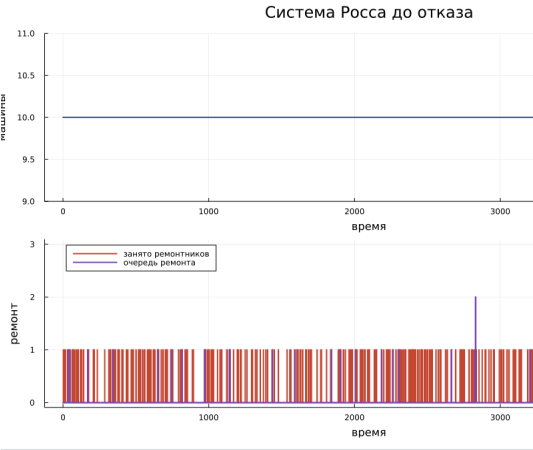
```
println(run_table[:, [:run, :failure_time, :utilization, :mean_queue, :transitions]])
println(comparison)
figure = reserve_trace_plot(first_trace, 10)
store_figure("reserve_baseline", "failure_path.png", figure)
figure
```

5x5 DataFrame

Row	run	failure_time	utilization	mean_queue	transitions
	Int64	Float64	Float64	Float64	Int64
1	1	4196.93	0.089566	0.00981281	1628
2	2	22311.6	0.101341	0.0111155	9172
3	3	66946.4	0.10103	0.0117991	26852
4	4	12198.4	0.0955477	0.00874825	4780
5	5	31867.1	0.102885	0.0116982	12948

1x6 DataFrame

Row	observed_mean	analytical_mean	observed_utilization	analytical_utilization	observed_queue
	Float64	Float64	Float64	Float64	Float64
1	27504.1	12340.0	0.099874	0.0996759	0.0996759



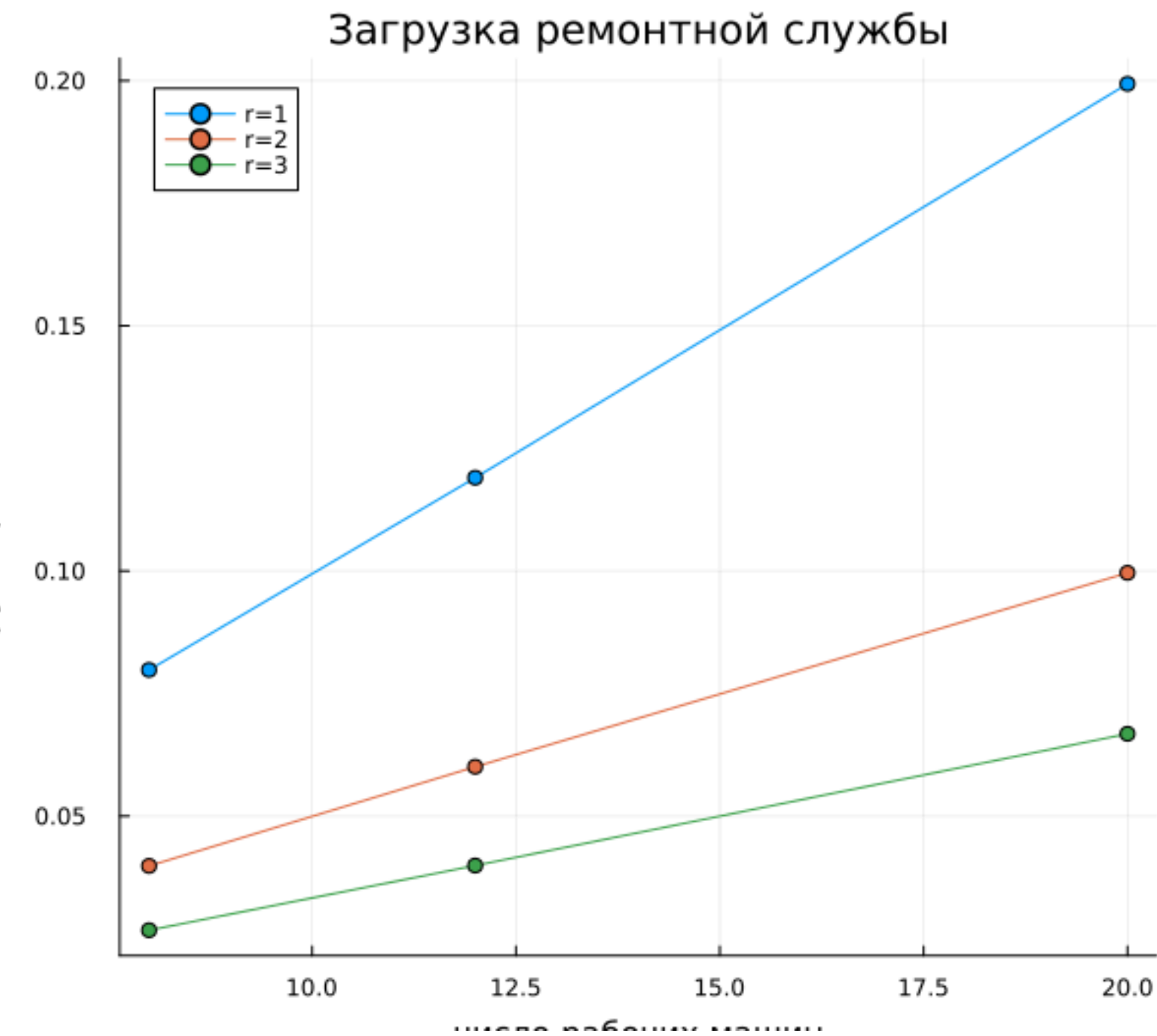
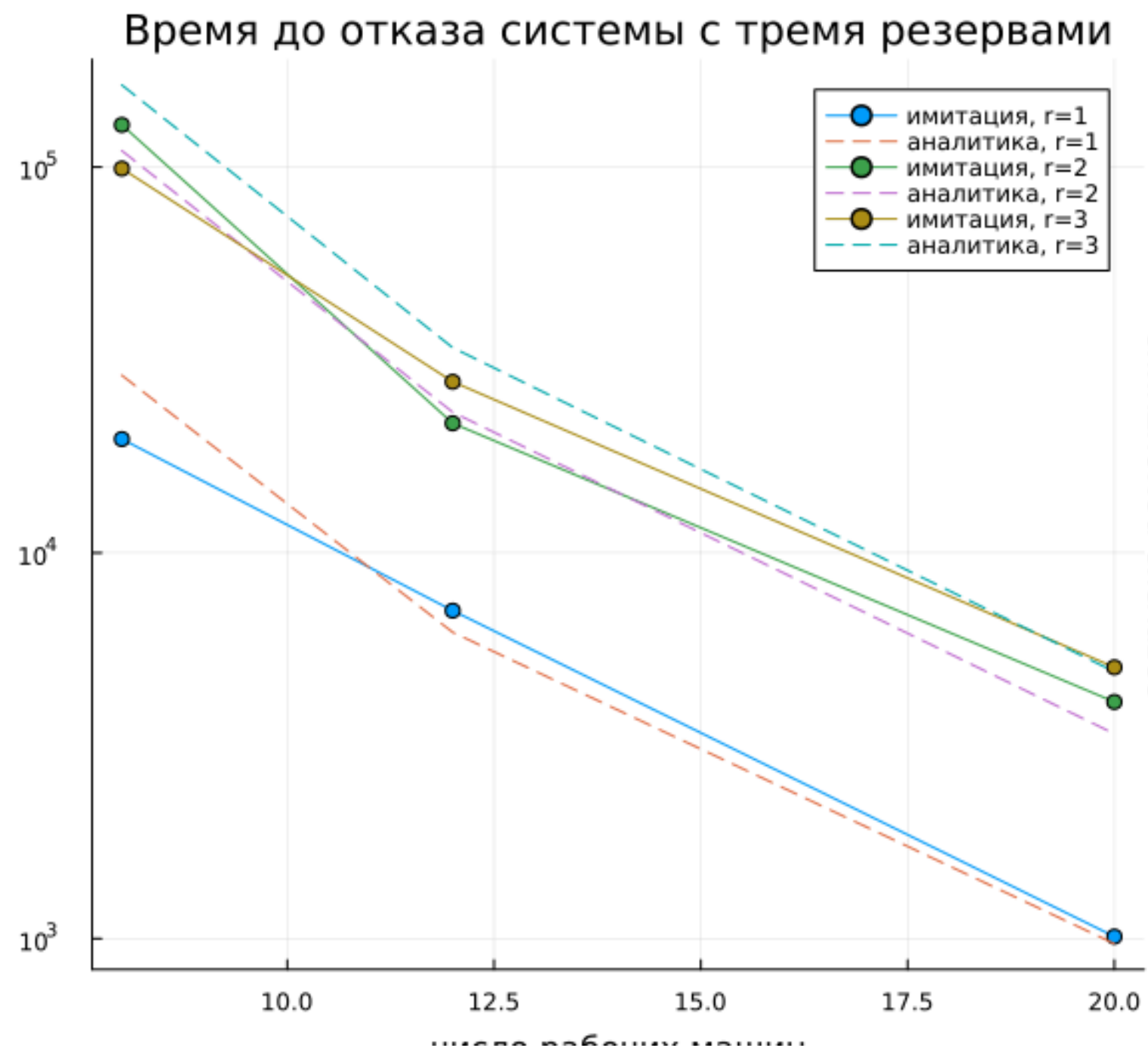
This notebook was generated using [Literate.jl](#).

Пять прогонов и рассчитанные показатели

4.11 Факторный план

- $N \in \{8, 12, 20\}$;
- $r \in \{1, 2, 3\}$;
- $S = 3$;
- 40 повторов на сочетание;
- 360 независимых траекторий.

4.12 Время отказа и загрузка



Имитация и аналитика

4.13 Эффект ремонтников

- увеличение r повышает время до отказа;
- загрузка одного ремонтника уменьшается;
- при трёх ремонтниках очередь ремонта равна нулю;
- при большем N отказы чаще, поэтому резерв исчерпывается быстрее.

Раздел 5

5 Проверка и результат

5.1 Набор проверок

- одинаковый seed воспроизводит таблицы;
- число занятых каналов не превышает c ;
- времена клиента упорядочены;
- ρ , W_q , L_q совпадают с контрольными формулами;
- отказ Росса происходит при $N - 1$ рабочих машинах;
- матричная невязка меньше 10^{-9} .

5.2 Форматы результатов

Для каждого из четырёх сценариев созданы:

- литературный исходник;
- чистый Julia-скрипт;
- выполненный Jupyter notebook;
- Quarto QMD;
- автономный HTML;
- CSV-таблицы и график.

5.3 Итоги

- М/М/с подтверждена формулой Эрланга С;
- закон Литтла проверен на стационарных выборках;
- система Росса проверена решением СТМС;
- исследованы несколько ремонтников и разные N ;
- результаты полностью воспроизводятся из [Manifest.toml](#) и [Makefile](#).